

LC 2812: Find Safest Path in a Grid

Target Companies: Wells Fargo, IMC

© Authored by Arpit Raj, LNMIIT Jaipur | July 23, 2026

Problem Understanding

Key variables:

0 = safe

1 = thief

"I need to be as far as thief as possible, while going from 0, 0 to n-1, m-1"

O/P → min (distances from thief during traversal)

Example Visualization



now assume aadya goes like:

2	1	2
1	0	1
2	1	2

then $\min(2, 1, 2, 1, 2) = 1 \rightarrow \text{O/P}$

we need the path whose minimum value is max

Algorithm Strategy

S1: run multi source BFS, with every source as a thief to make distance matrix

S2: now Aadya knows distances, can she now only step on cells having distance $\geq$ X

let's say check(4) impossible, then 5, 6, 7 also impossible

hence **BINARY SEARCH**

C++ Implementation

```
class Solution {
public:
    int n;
    int m;
    bool check(int mid, vector<vector<int>>& dist) {
        if(dist[0][0] < mid || dist[n-1][m-1] < mid) {
            return false; //these 2 cells were guaranteed to be in path
                           //and inka distance hi minimum se kam aa rha
        }

        vector<pair<int,int>> dir = {{1,0},{0,1},{-1,0},{0,-1}};
        queue<pair<int,int>> q;
        vector<vector<int>> seen(n, vector<int>(m, 0));

        seen[0][0] = 1;
        q.push({0,0});
        while(!q.empty()){
            int currsz = q.size();
            while(currsz--){
                int r = q.front().first;
                int c = q.front().second;
                if(r == n-1 && c == m-1) return true;
                q.pop();

                for(auto it : dir){
                    int nr = r + it.first;
                    int nc = c + it.second;

                    if(nr >= 0 && nc >= 0 && nr < n && nc < m){
                        if(!seen[nr][nc] && dist[nr][nc] >= mid){
                            seen[nr][nc] = 1;
                            q.push({nr,nc});
                        }
                    }
                }
            }
        }
        return false;
    }

    int maximumSafenessFactor(vector<vector<int>>& grid) {
        n = grid.size();
        m = grid[0].size();
        vector<pair<int,int>> dir = {{1,0},{0,1},{-1,0},{0,-1}};
        queue<pair<int,int>> q;
        vector<vector<int>> dist(n, vector<int>(m, -1));

        for(int i = 0; i < n; i++){ //preparing multisource bfs for
            distance matrix
```

```

        for(int j = 0; j < m; j++){
            if(grid[i][j] == 1){
                dist[i][j] = 0;
                q.push({i,j});
            }
        }
    }

    while(!q.empty()){ //running a multisource bfs for distance matrix
        int currsz = q.size();
        while(currsz--){
            int r = q.front().first;
            int c = q.front().second;
            q.pop();

            for(auto it : dir){
                int nr = r + it.first;
                int nc = c + it.second;

                if(nr >= 0 && nc >= 0 && nr < n && nc < m){
                    if(dist[nr][nc] == -1){
                        dist[nr][nc] = dist[r][c] + 1;
                        q.push({nr,nc});
                    }
                }
            }
        }
    }

    //now we will run a binary search to look for a value such that
    //it is the minimum in the whole path which can lead to n-1 and m-1
    //poore path ka minimum hi decide krega tumhari safety
    //maan lo poore paath ka minimum 5 he to path ke baaki elements
    should be >=5
    //maan le agar 5 eenough he to aur bada ka try kar to increase
    safety
    //agar 5 se possible nahi to 6,7,8 se bhi nahi hoga , kyunki agar 5
    ke liye 5,6,7....hi nhi thee
    //to 6 ke liye 6,7,8.....kaha se hi honge

    int l = 0; //min distance from thief
    int h = n + m; //max manhattan distance possible from a thief
    (corner to corner)
    int ans = 0;

    while(l <= h){
        int mid = l + (h - l) / 2;
        if(check(mid, dist)){ //check me basically bas dekhenge , ki
            //end tak reaach kr skte? while
            ensuring ki sirf >=mid pe hi step kare
            l = mid + 1;
            ans = mid;
        }
    }

```

```
        else{
            h = mid - 1;
        }
    }

    return ans;
}

};
```